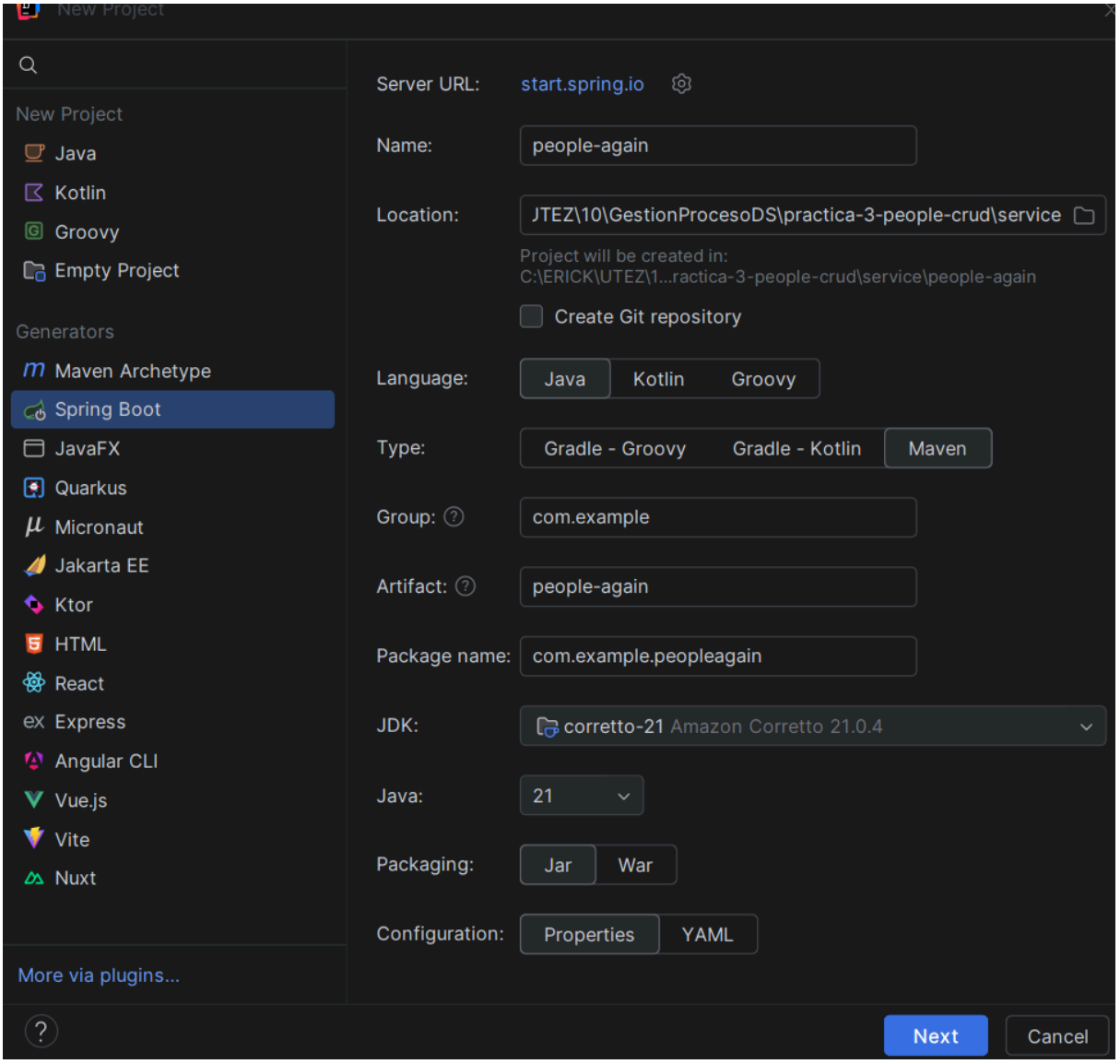
CRUD SpringBoot y Frontend



Al crear el proyecto dejar JDK amazon correto-21 y la version de java 21, no olvidar maven.

Damos Next, y en dependencias:

- Spring Web
- Spring Data JPA
- MySQL Driver

- Lombok

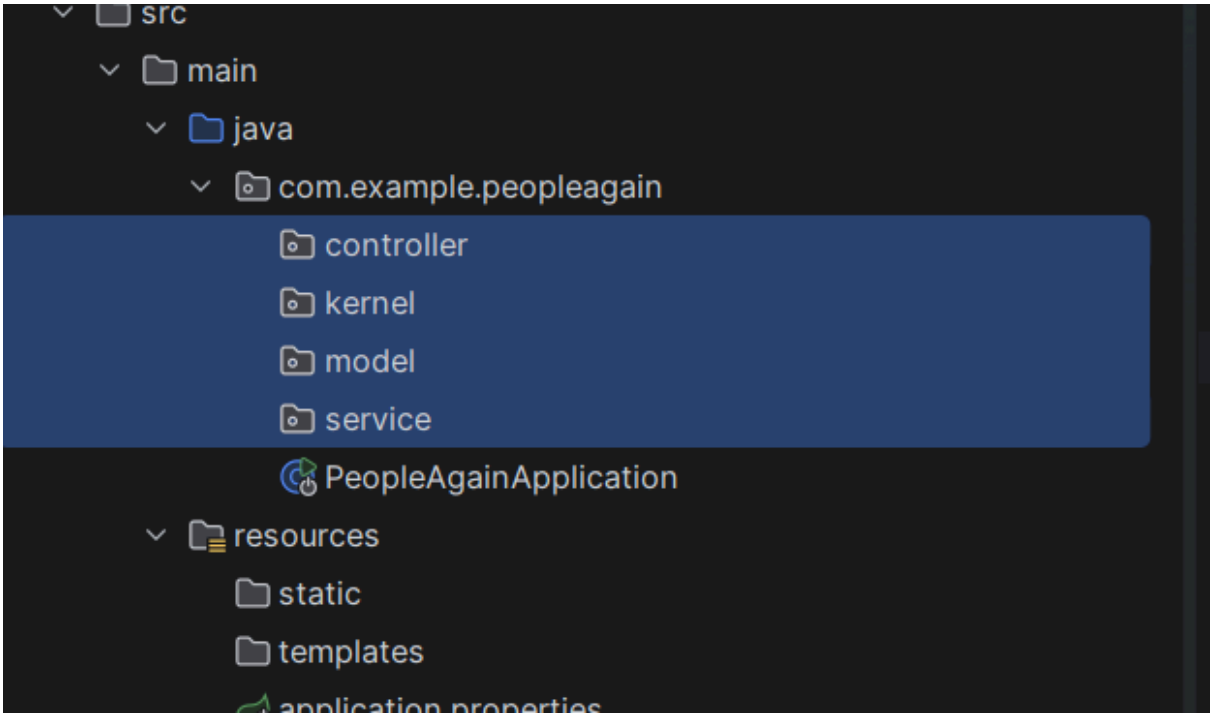
1.- Abrir properties y pegar:

```
# Conexión a MySQL. En local apunta a localhost.
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/
people-dev?createDatabaseIfNotExist=true&serverTimezone=UT
C}
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASS:root}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Drive
r

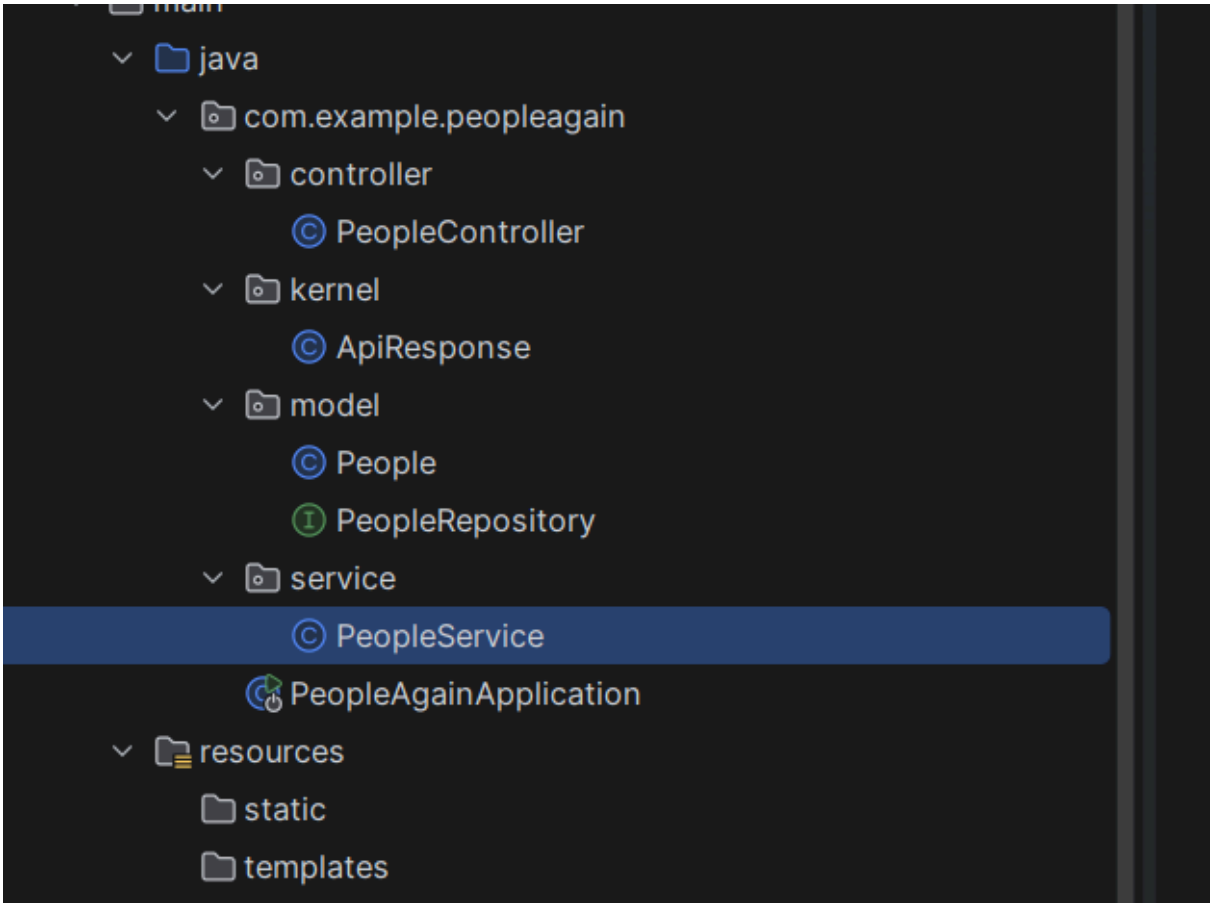
server.port=${SERVER_PORT:8080}

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.diale
ct.MySQLDialect
```

Crear estos 4 en el com.example:



Y sus respectivos archivos



2.- El modelo / Entity

```java
package com.example.peopleagain.model;

import jakarta.persistence.*;
import lombok.Data;

@Data                       // Lombok: crea getters, setters, toString, etc. solo
@Entity                     // Esto es una tabla de la BD
@Table(name = "people")     // La tabla se llamará "people"
public class People {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;   // una columna "nombre"
}
```

3.- El repository (se pega esto abajo del pack)

```java
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PeopleRepository extends JpaRepository<People, Long> {
}
```

4.- Luego ApiResponse

```java
import lombok.Data;
import org.springframework.http.HttpStatus;

@Data
public class ApiResponse {
```

```
    private String message;
    private Object data;
    private Boolean error;
    private HttpStatus status;

    public ApiResponse(String message, Object data, Boolean
error, HttpStatus status) {
        this.message = message;
        this.data = data;
        this.error = error;
        this.status = status;
    }
}
```

5.- Sigue el Service

```
import com.example.peopleagain.kernel.ApiResponse;
import com.example.peopleagain.model.People;
import com.example.peopleagain.model.PeopleRepository;

import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class PeopleService {

    @Autowired
    private PeopleRepository peopleRepository;


    public ApiResponse getAll() {
        List<People> people = peopleRepository.findAll();
```

```
        return new ApiResponse("Personas obtenidas", peopl
e, false, HttpStatus.OK);
    }


    public ApiResponse getOne(Long id) {
        Optional<People> p = peopleRepository.findById(id);
        if (p.isEmpty()) {
            return new ApiResponse("No existe esa persona",
null, true, HttpStatus.NOT_FOUND);
        }
        return new ApiResponse("Persona encontrada", p.get
(), false, HttpStatus.OK);
    }


    public ApiResponse crear(People people) {
        People saved = peopleRepository.save(people);
        return new ApiResponse("Persona creada", saved, fal
se, HttpStatus.CREATED);
    }

    public ApiResponse actualizar(Long id, People datos) {
        Optional<People> existente = peopleRepository.findB
yId(id);
        if (existente.isEmpty()) {
            return new ApiResponse("No existe esa persona",
null, true, HttpStatus.NOT_FOUND);
        }
        People p = existente.get();
        p.setNombre(datos.getNombre());
        People actualizado = peopleRepository.save(p);
        return new ApiResponse("Persona actualizada", actua
lizado, false, HttpStatus.OK);
    }

    public ApiResponse eliminar(Long id) {
        if (!peopleRepository.existsById(id)) {
```

```
            return new ApiResponse("No existe esa persona",
null, true, HttpStatus.NOT_FOUND);
        }
        peopleRepository.deleteById(id);
        return new ApiResponse("Persona eliminada", null, f
alse, HttpStatus.OK);
    }
}
```

6.- Controller

```
import com.example.peopleagain.kernel.ApiResponse;
import com.example.peopleagain.model.People;
import com.example.peopleagain.service.PeopleService;
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController                         // este controlador de
vuelve JSON
@RequestMapping("/people")
@CrossOrigin(origins = "*")
public class PeopleController {

    @Autowired
    private PeopleService peopleService;

    // GET http://localhost:8080/people
    @GetMapping
    public ResponseEntity<ApiResponse> listar() {
        ApiResponse r = peopleService.getAll();
        return ResponseEntity.status(r.getStatus()).body
(r);
    }

    // GET http://localhost:8080/people/1
```

```
    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse> obtener(@PathVariabl
e Long id) {
        ApiResponse r = peopleService.getOne(id);
        return ResponseEntity.status(r.getStatus()).body
(r);
    }

    // POST http://localhost:8080/people   body: { "nombr
e": "Erick" }
    @PostMapping
    public ResponseEntity<ApiResponse> crear(@RequestBody P
eople people) {
        ApiResponse r = peopleService.crear(people);
        return ResponseEntity.status(r.getStatus()).body
(r);
    }

    // PUT http://localhost:8080/people/1   body: { "nombr
e": "Nuevo" }
    @PutMapping("/{id}")
    public ResponseEntity<ApiResponse> actualizar(@PathVari
able Long id, @RequestBody People people) {
        ApiResponse r = peopleService.actualizar(id, peopl
e);
        return ResponseEntity.status(r.getStatus()).body
(r);
    }

    // DELETE http://localhost:8080/people/1
    @DeleteMapping("/{id}")
    public ResponseEntity<ApiResponse> eliminar(@PathVariab
le Long id) {
        ApiResponse r = peopleService.eliminar(id);
        return ResponseEntity.status(r.getStatus()).body
(r);
    }
}
```

Frontend

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, init
ial-scale=1.0" />
    <title>PeopleCRUD</title>
  </head>
  <body>
    <h1>PeopleCRUD</h1>

    <form>
      <input type="text" id="input1" placeholder="Ingresa u
n nombre" />
      <button id="btn" type="submit">Agregar</button>
    </form>

    <ul id="ul"></ul>

    <script src="app2.js"></script>
  </body>
</html>
```

JS con solo create

```
"use strict";

const d = document, c = console;

// Debemos cambiar localhost por la ip publica cuando lo su
bamos a la ec2.
  // Debemos cambiar localhost por la ip publica cuando lo
subamos a la ec2.
  const LOCAL = "http://localhost:8080/people";
  const PROD  = "http://100.53.126.240:8089/people";
```

```
  const url = PROD;

const $btn = d.getElementById("btn");
const $ul = d.getElementById("ul");
const $input1 = d.getElementById("input1");


function cargarDatos() {

    fetch(url)
    .then(res => res.json())
        .then(data => {
            $ul.innerHTML = "";
            const dataClean = data.data;
            dataClean.forEach(dato => agregar(dato));
        })
        .catch(err => c.error(err));
}

// ____________________________________________________________
function agregar(dato) {
    const li = d.createElement("li");
    li.textContent = dato.nombre;
    $ul.appendChild(li);
}

// Agregar al dar click
$btn.addEventListener("click", (e)=> {
    e.preventDefault();

    if(!$input1.value.trim()) return;

    fetch(url, {
        method: "POST",
        headers: {
            "Content-type": "application/json"
        },
        body: JSON.stringify({nombre: $input1.value})
```

```
    })
    .then(res => res.json())
    .then(data => {
        c.log(data);
        cargarDatos();
        $input1.value = "";
    })
    .catch(err => c.error(err));
});

cargarDatos();
```

JS con crud completo

```
 "use strict";

  const d = document, c = console;

  // Debemos cambiar localhost por la ip publica cuando lo
subamos a la ec2.
  const url = "http://localhost:8080/people";

  const $btn = d.getElementById("btn");
  const $ul = d.getElementById("ul");
  const $input1 = d.getElementById("input1");

  // Guarda el id que estamos editando. null = estamos crea
ndo.
  let editandoId = null;


  function cargarDatos() {

      fetch(url)
      .then(res => res.json())
          .then(data => {
              $ul.innerHTML = "";
              const dataClean = data.data;
```

```
                dataClean.forEach(dato => agregar(dato));
            })
            .catch(err => c.error(err));
    }

    // ____________________________________________________________
_
    function agregar(dato) {
        const li = d.createElement("li");
        li.textContent = dato.nombre + " ";

        // Boton editar
        const btnEditar = d.createElement("button");
        btnEditar.textContent = "Editar";
        btnEditar.addEventListener("click", () => editar(dat
o));

        // Boton borrar
        const btnBorrar = d.createElement("button");
        btnBorrar.textContent = "Borrar";
        btnBorrar.addEventListener("click", () => borrar(dat
o.id));

        li.appendChild(btnEditar);
        li.appendChild(btnBorrar);
        $ul.appendChild(li);
    }

    // Pasar al modo edicion: rellena el input y recuerda el
id
    function editar(dato) {
        $input1.value = dato.nombre;
        editandoId = dato.id;
        $btn.textContent = "Actualizar";
    }

    // ____________________________________________________________
_
```

```
  // Borrar al dar click (DELETE)
  function borrar(id) {
      fetch(url + "/" + id, {
          method: "DELETE"
      })
      .then(res => res.json())
      .then(data => {
          c.log(data);
          cargarDatos();
      })
      .catch(err => c.error(err));
  }

  // ____________________________________________________________
_
  // Crear o Actualizar al dar click (POST o PUT)
  $btn.addEventListener("click", (e)=> {
      e.preventDefault();

      if(!$input1.value.trim()) return;

      // Si hay un id en edicion -> PUT. Si no -> POST.
      const metodo = editandoId ? "PUT" : "POST";
      const endpoint = editandoId ? url + "/" + editandoId
: url;

      fetch(endpoint, {
          method: metodo,
          headers: {
              "Content-type": "application/json"
          },
          body: JSON.stringify({nombre: $input1.value})
      })
      .then(res => res.json())
      .then(data => {
          c.log(data);
          cargarDatos();
          $input1.value = "";
```

```
            editandoId = null;          // salimos del modo e
dicion
            $btn.textContent = "Agregar";
        })
        .catch(err => c.error(err));
    });

    cargarDatos();
```

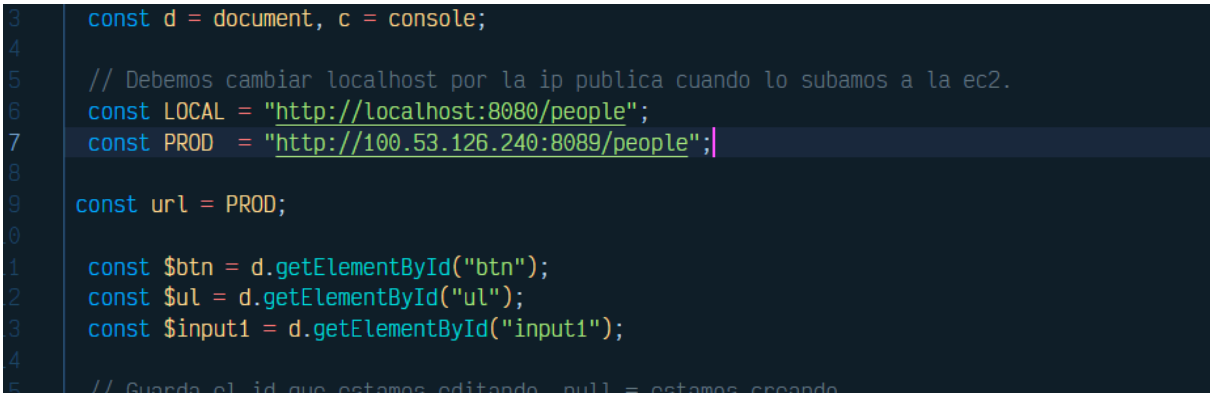
Lo de Docker

Comandos de manual

```
docker ps                    # contenedores corriendo
docker ps -a                 # TODOS (incluidos apagados)
docker images                # imágenes que tienes
docker logs -f pmng-back     # ver logs en vivo (Ctrl+C par
a salir, el contenedor sigue)
docker restart pmng-back     # reiniciar
docker stop pmng-back        # detener
docker rm pmng-back          # borrar (debe estar detenido)
docker rmi backend:v1.0      # borrar una imagen
docker network ls            # ver redes
docker exec -it pmng-mysql bash   # entrar DENTRO de un con
tenedor
```

1.- URLS

Una vez ya testeamos que funcione en local. Cambiamos la URL del backend por la direccion publica. Recordar cambiar la ipv4 y el puerto externo por el que se expondrá el backend.



```
const d = document, c = console;

// Debemos cambiar localhost por la ip publica cuando lo subamos a la ec2.
const LOCAL = "http://localhost:8080/people";
const PROD  = "http://100.53.126.240:8089/people";

const url = PROD;

const $btn = d.getElementById("btn");
const $ul = d.getElementById("ul");
const $input1 = d.getElementById("input1");

// Guarda el id que estamos editando, null = estamos creando
```

Solo cambiar url a PROD en vez de local.

2.- Dockerfiles

Asegurarnos de escribir Dockerfile y no DockerFile

Para el backend si siempre usamos 21 y jdk amazon coretto 21

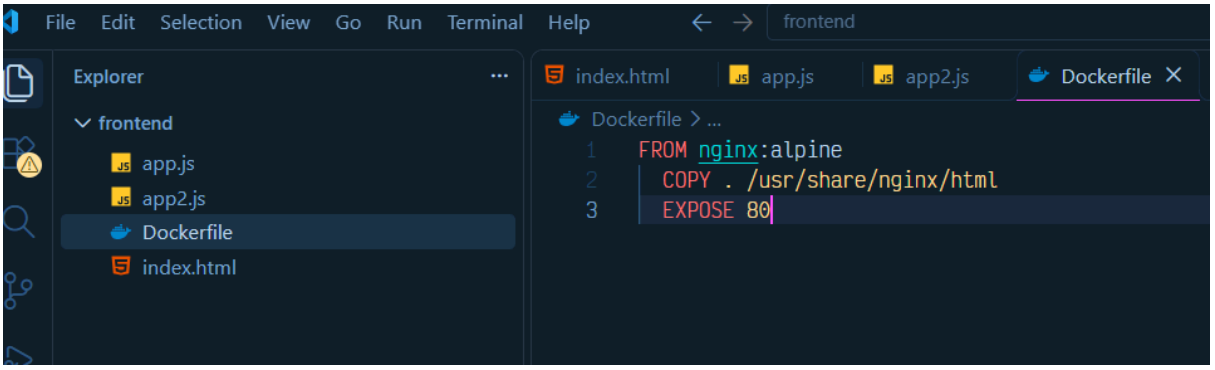
```
# ===== ETAPA 1: compilar =====
  FROM maven:3.9-eclipse-temurin-21 AS build
  WORKDIR /app
  COPY pom.xml .
  RUN mvn dependency:go-offline -B
  COPY src ./src
  RUN mvn clean package -DskipTests

  # ===== ETAPA 2: ejecutar =====
  FROM eclipse-temurin:21-jre
  WORKDIR /app
  COPY --from=build /app/target/*.jar app.jar
  EXPOSE 8080
  ENTRYPOINT ["java", "-jar", "app.jar"]
```

Para el frontend con esto es suficiente.

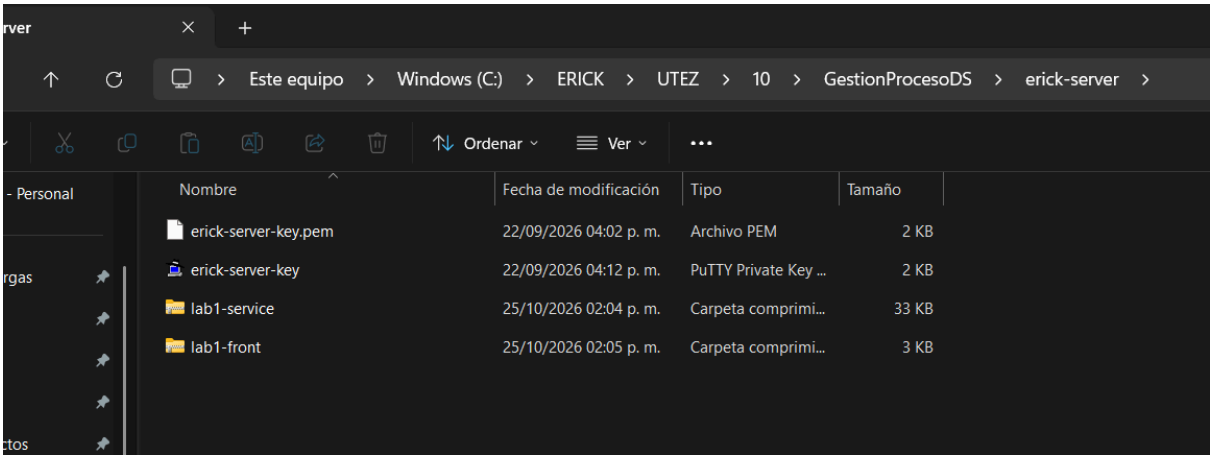
```
FROM nginx:alpine
  COPY . /usr/share/nginx/html
```

```
EXPOSE 80
```



3.- zips y subir archivos.

Cuando comprimamos los zips los movemos a donde esta la pem.



Abrimos en la terminal

```
  scp -i "erick-server-key.pem" NOMBREARCHIVO.zip ubuntu@ec
2-100-53-126-240.compute-1.amazonaws.com:/home/ubuntu/
```

Ese seria el comando de siempre.

Lo que siempre cambia es el host que va desde el @hasta los :

Es algo asi: ec2-100-53-126-240.compute-1.amazonaws.com



Y ya nos podemos conectar con ssh para verificar.

Ya que estan subidos.



4.- Hacemos unzip

de cada archivo y ya se descargan con el nombre con el que definimos el proyecto.

En esta caso es frontend y people-again

Podemos hacer docker —version para ver que este bien.

5.- Creamos la red

```
docker network create pmng-net
```

Con docker network ls podemos ver que ya esta la red.

6.- Correr contenedores

MYSQL

```
docker run -d \
    --name pmng-mysql \
    --network pmng-net \
    -e MYSQL_ROOT_PASSWORD=root \
    -e MYSQL_DATABASE=people-dev \
```

```
    -p 3306:3306 \
    mysql:8.0
```

IMPORTANTE: El —name del contenedor es el que tenemos que poner al correr el backend. Igual con la base de datos.

Si ya existia podemos hacer docker rm idContainer

BACKEND

PRIMERO HAY QUE CONSTRUIR LA IMAGEN Y LUEGO LEVANTAR EL CONTENEDOR

ENTRAMOS A LA CARPETA DESCOMPRIMIDA Y HACEMOS:

```
docker build -t backend:v1.0 .
```

Luego con docker images podemos revisar que ya se haya creado.

LO SIGUIENTE ES LA EJECUCIÓN.

Aquí es donde es importante poner el NOMBRE DEL CONTENEDOR de mysql que levantamos.

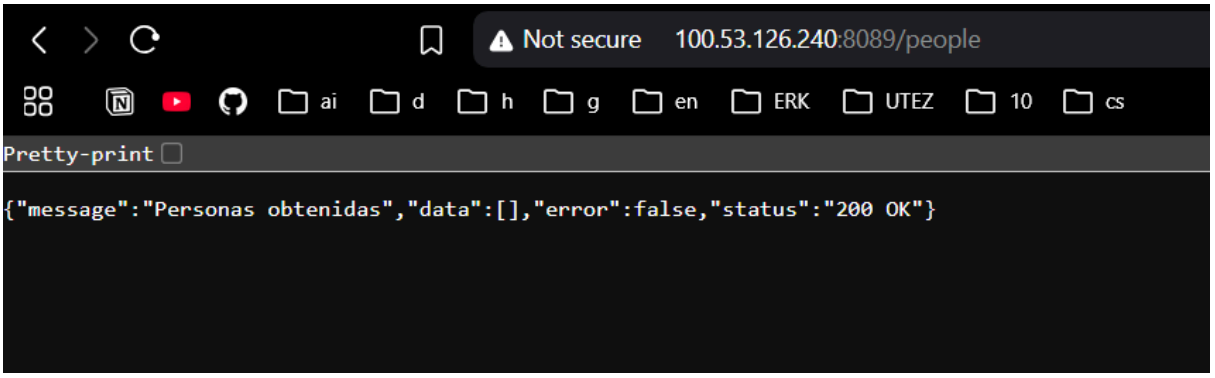
Y cambiar la base de datos con la que levantamos el servicio tambien.

```
 docker run -d \
    --name pmng-back \
    --network pmng-net \
    -e DB_URL="jdbc:mysql://pmng-mysql:3306/people-dev?crea
teDatabaseIfNotExist=true&serverTimezone=UTC" \
    -e DB_USER=root \
    -e DB_PASS=root \
    -p 8089:8080 \
    backend:v1.0
```

Parte	Significado	Debe coincidir con
`-d`	corre en segundo plano	—
`--name pmng-back`	nombre de este contenedor	—
`--network pmng-net`	lo mete a tu red	la misma red de MySQL
`-e DB_URL= ... pmng-mysql ...`	sobreescribe el `localhost` del properties	el `--name` de MySQL (`pmng-mysql`) y la BD (`people-dev`)
`-e DB_USER=root`	usuario de la BD	el user de MySQL (root)
`-e DB_PASS=root`	contraseña	`MYSQL_ROOT_PASSWORD=root` de MySQL
`-p 8089:8080`	afuera 8089, adentro 8080	el `:8089` de la URL en tu `app.js`
`backend:v1.0`	la imagen que construiste	el `-t` del build

Tambien ojo con el puerto externo que exponemos y la imagen.

Y ahi ya esta levantado:



FRONTEND

PRIMERO HABRÍA QUE CONSTRUIR LA IMAGEN. (nota si hace falta cambiar algo hacer nano app.js)

```
docker build -t frontend:v1.0 .
```

y ahora run

```
docker run -d \
   --name pmng-front \
   -p 5173:80 \
   frontend:v1.0
```

OJO: AQUI NO CONECTAMOS EL FRONTEND A LA NETWORK PORQUE EN EL DIAGRAMA NO ESTABA ASÍ.